

Enhanced Promptomatix: An Automatic Prompt Optimization Framework for Large Language Models

Xuan Thao Dao^{1*}, Phu Thai Nguyen^{1*}, Manh Phong Duong^{1*}
and Quang Vinh Nguyen^{1*}

^{1*}University of Engineering and Technology, Vietnam National
University Hanoi.

*Corresponding author(s). E-mail(s): 23020156@vnu.edu.vn;
23020153@vnu.edu.vn; 23020132@vnu.edu.vn; 21020264@vnu.edu.vn;

Abstract

Large language models (LLMs) show that output varies considerably depending on how prompts are written. However, designing good prompts in practice remains time-consuming and heavily reliant on personal experience. This research focuses on improving the Promptomatix [1] system to reduce manual work in the prompt optimization process. Specifically, the system is supplemented with a task classification step to determine whether the input is based on reference data. This classification helps select the appropriate optimization approach instead of applying a single process to all cases. In addition, the prompt quality assessment method is adjusted by using various criteria combined with semantic similarity measurements. This helps to more accurately reflect the output compared to the previous simple scoring method. Experimental results show that the improved version provides more stable and consistent output compared to the original Promptomatix simple meta prompt backend, especially in complex tasks.

1 Introduction

Recent large language models (LLMs) have demonstrated a remarkable adaptability across multiple domains. These LLMs are implemented using user prompts as instructions, therefore the quality of the prompt has a significant impact on the

⁰Code and experiments available at: <https://github.com/DXT0509/promptomatix>

model’s performance. Effective rapid response techniques have evolved and become more sophisticated. However, prompt engineering’s scalability and accessibility remain significant restrictions for widespread LLM adoption in practical applications. Systematic evaluation of prompt effectiveness is very costly in terms of resources and time. Reproducibility and fair comparison are limited by the lack of standardized protocols. Manual prompt engineering fails to scale in various tasks and domains, most optimization methods require task-specific large datasets.

To handle all these limitations, Promptomatix was introduced, which combines meta-learning and cost-aware strategies to analyze user intent, generates synthetic training data, selects effective prompting techniques and iteratively refines prompts based on feedback and performance. Promptomatix does not require access to the model’s internal gradients, parameters and state, thus it possesses Black-Box Compatibility. However, the result of this method could be affected by model’s behavior.

Motivated by several restrictions of Promptomatix, we made a contribution on the backend: simple meta prompt of the original Promptomatix paper. Our method introduces prompt type classification, dual evaluation strategies, few-shot prompting, and beam search-based optimization to improve both robustness and effectiveness.

Our contributions are as follows:

- We introduce data-dependent and data-independent evaluation methods tailored to reference-based and zero-reference prompts.
- We develop a task classification agent using meta-prompts to automatically select the appropriate evaluation mechanism.
- We further enhance evaluation quality by adopting semantic similarity with MPNet and a weighted LLM-as-a-judge scheme.
- We integrate few-shot prompting with prompt beam search to improve the robustness and effectiveness of prompt optimization.

2 Related Work

Currently, prompts are widely used to adapt large-scale programming language models for specific problems without requiring rework. Prompts were previously written manually. This technique relies heavily on individual experience and often requires significant time for experimentation. Furthermore, results can change significantly with even slight modifications to the writing style. This makes rapid, large-scale manual application difficult.

Several studies have focused on optimizing automated prompts to address this problem. AutoPrompt [2] is considered a token-finding problem. Newer methods, such as ProTeGi [3], use iterative processes to modify model-based messages. However, most existing methods apply a general evaluation mechanism to all tasks, which is direct comparison with the ground truth, leading to a lack of accuracy in evaluating zero-reference prompts.

3 Background

3.1 Reference-based Prompt

Reference-based prompt is a data-dependent prompt because the measured quality depends directly on the dataset’s labels and what the dataset considers “correct”. A **reference-based** evaluation uses g_i to score the model response R_i . This setting is suitable for tasks where a ground-truth answer exists (QA with labeled data,...). Here is the scoring function of a reference-based evaluation:

$$s_i = M(R_i, g_i), \quad \text{Score}(\mathcal{D}) = \frac{1}{N} \sum_{i=1}^N s_i, \quad (1)$$

3.2 Zero-reference Prompt

A **zero-reference** is a data-independent prompt which does not use g_i . It is used when there are no gold answers or they are ambiguous. So scoring relies on alternative signals, such as: format checks, length limits, consistency, checks across multiple samples, an LLM-as-a-judge without references. Here is the zero-reference prompt scoring function:

$$s_i = \widetilde{M}(x_i, R_i), \quad \text{Score}(\mathcal{D}) = \frac{1}{N} \sum_{i=1}^N s_i. \quad (2)$$

Because of the lack of references, the score could favor responses that sound correct but are wrong or lacking.

3.3 Semantic Similarity Metric

Semantic similarity measures whether a response R and a gold answer G convey the same meaning, even if they use different words. Using an embedding model E , the response and gold answer are mapped into vectors and then cosine similarity is computed:

$$\text{Score}_{\text{semantic}}(R, G) = \frac{E(R) \cdot E(G)}{\|E(R)\| \|E(G)\|}. \quad (3)$$

This metric is often more tolerant than n-gram overlap metrics (ROUGE/BLEU) because it focuses on meaning and context rather than exact word matching. However, it may still assign a high score to responses that are semantically close but contain factual mistakes.

3.4 Scoring with LLM-as-a-judge

LLM-as-a-judge uses a separate LLM to evaluate the quality of a response based on a rubric (relevance, correctness, completeness, clarity,...). It can output:

- a scalar score (1–5, 0–10,...);

- a pairwise preference between two candidates.

We can write:

$$s_i = J(x_i, R_i, g_i), \quad (4)$$

where g_i is optional:

- **Reference-based judging:** $J(x_i, R_i, g_i)$ compares R_i against the gold answer.
- **Reference-free judging:** $J(x_i, R_i)$ evaluates R_i using only the input.

The outputs can be unstable and may repeat runs to improve reliability.

3.5 Baseline: Zero-shot Prompting

Zero-shot prompting means that the prompt used to interact with the model won't contain examples or demonstrations. The zero-shot prompt directly instructs the model to perform a task without any additional examples to steer it.

The original Promptmatix method uses zero-shot prompting to optimize the prompt – that is, it only sends constraints and instructions for optimizing the prompt but not the actual context of the dataset, resulting in a low-quality optimized prompt that doesn't closely adhere to the dataset's objectives.

3.6 Few-shot Prompting

Few-shot prompting is a prompting method where we include a small number of input-output pairs inside the prompt to guide the LLM on what to do and how to format the answer. Instead of training the model, we provide demonstrations directly in the prompt. Given k examples $\{(x_j, y_j)\}_{j=1}^k$, the prompt includes these demonstrations before the test input:

$$p = [(x_1 \rightarrow y_1), \dots, (x_k \rightarrow y_k), x_{\text{test}} \rightarrow ?], \quad R = \text{LLM}(p, x_{\text{test}}). \quad (5)$$

Few-shot prompting can reduce ambiguity and improve consistency, but it increases prompt length and token cost.

4 Propose Method

4.1 Overview of the Proposed Method

To address the critical shortcomings of the original Promptmatix, we propose our methods.

The system calls an LLM to classify prompts into zero-reference or reference-based prompts, then leverages the data already present in the training data, using few-shot prompting to provide context for the LLM, and then uses Beam Search to expand the prompt search space, aiming to obtain the prompt with the best score among the different prompt variations.

The overall process of the method: the system starts by receiving a prompt to be optimized. This prompt will be classified into one of two types as shown in the figure 1. Depending on the type of prompt, there is a separate scoring mechanism and

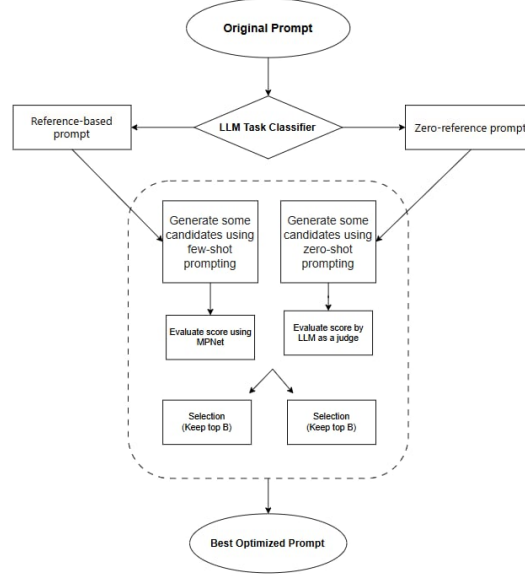


Fig. 1 Overview The Proposed Prompt Optimization Framework

a separate mechanism for generating optimal prompts as candidates. Consider the original prompt as one candidate. Each candidate generates several candidates. Finally, select B candidates with the highest scores and start a new beam search loop. The final result will be the prompt with the highest score among all generated candidates.

4.2 Task Classification Agent

We’ve developed a meta-prompt to guide the LLM in classifying prompts into one of two types: Reference-Based or Zero-Reference. By sending this meta-prompt to the LLM, the LLM will return one of two values: True or False. This classification helps the system determine whether the prompt will be evaluated using the LLM-as-a-judge method or MPNet.

$$\text{Route}(P) = \begin{cases} \text{Reference-based Prompt} & \text{if Classifier}(P) = \text{True} \\ \text{Zero-reference Prompt} & \text{if Classifier}(P) = \text{False} \end{cases} \quad (6)$$

4.3 Evaluation method

As mentioned, prompts are divided into two types. Therefore, the evaluation methods will differ for each type.

1. With a reference-based prompt, if the prompt’s intent relates to task classification, we use EM metrics for evaluation. For other tasks, we use all-mpnet-base-v2, the best performing model from the sentence Transformers [4] library to compare semantic similarity between the LLM’s response and Ground Truth in each test data set, instead of using BERTscore [5] as in the Promptomatix paper, which does

not compare based on semantic similarity. The final score of the prompt will be the average of the scores from all tests.

2. With a zero-reference prompt, the prompt is evaluated by taking the response of the LLM and assessing the quality of that response to the original prompt. Therefore, if the LLM’s response is good, the prompt is also highly rated. We have prepared a meta-prompt to guide the LLM in evaluating the quality of the response to the prompt based on five criteria: relevance, accuracy, consistency, readability_coherence, and efficiency. Each criterion receives a score between 0 and 1, with weights of: "relevance": 0.30, "accuracy": 0.30, "consistency": 0.10, "readability_coherence": 0.15, "efficiency": 0.15. These are five criteria we referenced from previous studies on prompt evaluation. [6].

$$S_{total} = w_{rel}S_{rel} + w_{acc}S_{acc} + w_{con}S_{con} + w_{coh}S_{coh} + w_{eff}S_{eff} \quad (7)$$

where:

- $S_{rel}, S_{acc}, S_{con}, S_{coh}, S_{eff}$: Scores for Relevance, Accuracy, Consistency, Readability_Coherence, and Efficiency.
- w_i : Weights of each criterion, $\sum w_i = 1$.

4.4 Optimization method

We have contributed the following improvements to the original method in terms of prompt optimization.

1. Using few-shot prompting allows the LLM to understand how it needs to respond to specific questions, thereby optimizing the prompt to work correctly within the dataset context, instead of using zero-shot prompting as a baseline.
2. Using beam search to optimize prompts: Instead of using greedy search, our method applies the beam search algorithm to expand the prompt search space, with beam width B. Specifically, this algorithm works as follows:

Initially, the LLM is required to generate k optimal prompts $\{C_1, C_2, \dots, C_k\}$ from the original prompt. The meta prompt force LLM to generate candidates sufficiently different to follow different beam directions. Keep B prompts with the highest score. Next, for each iteration, each candidate C_i generate another k optimal prompts, resulting $B \times k$ new candidates, group all old and new candidates together, select the B prompts with the highest scores to keep.

This process is repeated until the required number of cycles is reached. Finally, the best-evaluated prompts are selected as the final optimal prompt.

Using few-shot prompting in combination with beam search can help find the optimal prompt more effectively, but it also significantly increases the cost and time required to find the optimal prompt, depending on the number of candidates generated from a candidate, the number of candidates retained after each round, and the number of beam search rounds.

5 Experiments

We will evaluate the effectiveness of the method through three experimental phases, corresponding to three research questions:

- **RQ1:** How accurate is the task classification agent in classifying prompts into two types: zero-reference and reference-based?
- **RQ2:** How is the zero-reference prompt improved after optimization (evaluated across 5 criteria and win rate)?
- **RQ3:** The effectiveness of using few-shot + beam search for optimizing reference-based prompts.

5.1 Datasets

In the first testing phase: Testing the ability to classify prompts into zero-reference and reference-based prompts, we will use an **alpaca-cleaned** dataset [7]. This dataset provides instructions and input; for zero-reference prompts, the input will be empty, while the input will be valid for the other prompt types.

In the second testing phase, we use instructions in the **WizardLM_evol_instruct_70k** dataset [8] for the experiments.

In the final testing phase, we conducted experiments on four different tasks, corresponding to four sets of datasets:

- **AG News** for classification task [9]
- **XSum** for summarization task [10]
- **GSM8K** for math task [11]
- **squad_v2** for QA task [12, 13]

5.2 Setup

We used the gpt-oss-20b model for all tasks, except for the LLM-as-a-judge task, where we used the gpt-oss-120b model to improve evaluation accuracy.

For the first testing phase, we selected 500 examples. In the second testing phase, we selected 20 examples, and we only used greedy search and not beam search, even though we could have, due to time and cost reasons, to demonstrate that the meta prompt used to guide the optimization of our prompt worked effectively. In the final testing phase:

- With classification task, we selected 30 examples for training set and 300 examples for testing set, using EM metrics for evaluation.
- With summarization task, we selected 7 examples for training set and 50 examples for testing set, using MPNet for evaluation.
- With math and QA tasks, we selected 10 examples for training set and 50 examples for testing set, using MPNet for evaluation.

In the final testing phase, we will test every prompt on two separate runs with beam search, each using a different training set and testing set, and the evaluation result will be the average of the two runs.

We conducted this experiment using the following configuration: temperature = 0.0, top-p = 0.1, beam_size B = 2, k = 2, running 2 beam search loops.

5.3 Performance Results

5.3.1 Answer to RQ1

Table 1 shows the results for the first testing phase, revealing an accuracy rate of up to 90.6%. This demonstrates that in most cases, the prompts can be correctly classified, leading to the finding of appropriate metrics.

5.3.2 Answer to RQ2

In **Table 2**, we compare the average scores of all criteria between the original prompt and the optimized prompt. The results show that all criteria have improved. In particular, the efficiency criterion improved most significantly, by 11.7%, indicating that the optimized prompt is very effective at removing unnecessary details. The win rate of 75% also shows that in most cases, the prompt has improved in a positive direction.

5.3.3 Answer to RQ3

The experiment in **Table 3** is the most important, demonstrating the ability to optimize prompts for our method using few-shot prompting and beam search. Most tasks showed significant improvement, such as the classification task which improved by 202.9%, but the most impressive improvement was in the math task, which improved by 478.4%. However, the summarization task only improved by a mere 2.5%, because with this task, the user intent could be clearly expressed in the original prompt, even though it was very simple, and it also fit the dataset.

Table 1 Performance of Task
Classification Agent.

Metric	Value
Total Test Samples	500
Correct Classifications	453
Incorrect Classifications	47
Accuracy	90.6%

6 Conclusion

This study aimed to improve recommendation optimization efficiency by introducing an innovative method based on the simple meta prompt backend of Promptmatix. We propose further improvements to BeamSearch to optimize prompt search capabilities, use few-shot prompting to address the issue of prompts not performing correctly for specific dataset tasks, and suggest using metrics evaluated by LLM-as-a-judge, which helps to provide more accurate scores for each type of prompt.

Table 2 Performance for Zero-reference Prompts.

Criteria	Original Prompt	Optimized Prompt	Improvement (%)
Relevance	0.8785	0.9315	+6.0%
Accuracy	0.9135	0.9510	+4.1%
Consistency	0.8945	0.9460	+5.8%
Readability_Coherence	0.9065	0.9395	+3.6%
Efficiency	0.8035	0.8975	+11.7%
Weighted Score	0.8836	0.9349	+5.8%
Win Rate	-	-	75%

Table 3 Performance for Reference-based Prompts.

Task Category	Baseline (Promptmatix)	Proposed Method	Improvement (%)
Classification	0.2883	0.8733	+202.9%
Summarization	0.5787	0.5929	+2.5%
QA	0.2240	0.7463	+233.2%
Math	0.1642	0.9498	+478.4%

References

- [1] Murthy, R., Zhu, M., Yang, L., Qiu, J., Tan, J., Heinecke, S., Xiong, C., Savarese, S., Wang, H.: Promptmatix: An Automatic Prompt Optimization Framework for Large Language Models (2025). <https://arxiv.org/abs/2507.14241>
- [2] Shin, T., Razeghi, Y., IV, R.L.L., Wallace, E., Singh, S.: AutoPrompt: Eliciting knowledge from language models with automatically generated prompts. In: Empirical Methods in Natural Language Processing (EMNLP) (2020)
- [3] Pryzant, R., Iter, D., Li, J., Lee, Y., Zhu, C., Zeng, M.: Automatic prompt optimization with “gradient descent” and beam search. In: Bouamor, H., Pino, J., Bali, K. (eds.) Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp. 7957–7968. Association for Computational Linguistics, Singapore (2023). <https://doi.org/10.18653/v1/2023.emnlp-main.494> . <https://aclanthology.org/2023.emnlp-main.494/>
- [4] Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks (2019). <https://arxiv.org/abs/1908.10084>
- [5] Zhang, T., Kishore, V., Wu, F., Weinberger, K.Q., Artzi, Y.: BERTScore: Evaluating Text Generation with BERT (2020). <https://arxiv.org/abs/1904.09675>
- [6] Portkey: Evaluating Prompt Effectiveness: Key Metrics and Tools. <https://portkey.ai/blog/evaluating-prompt-effectiveness-key-metrics-and-tools/> (2023)

- [7] Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P., Hashimoto, T.B.: Stanford Alpaca: An Instruction-following LLaMA model. GitHub (2023)
- [8] Xu, C., Sun, Q., Zheng, K., Geng, X., Zhao, P., Feng, J., Tao, C., Lin, Q., Jiang, D.: WizardLM: Empowering large pre-trained language models to follow complex instructions (2025). <https://arxiv.org/abs/2304.12244>
- [9] Hugging Face Datasets: sh0416/ag_news: AG News Classification Dataset. https://huggingface.co/datasets/sh0416/ag_news. Dataset card on Hugging Face (2023)
- [10] Narayan, S., Cohen, S.B., Lapata, M.: Don’t give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization. ArXiv **abs/1808.08745** (2018)
- [11] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., Schulman, J.: Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168 (2021)
- [12] Rajpurkar, P., Zhang, J., Lopyrev, K., Liang, P.: SQuAD: 100,000+ questions for machine comprehension of text. In: Su, J., Duh, K., Carreras, X. (eds.) Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, pp. 2383–2392. Association for Computational Linguistics, Austin, Texas (2016). <https://doi.org/10.18653/v1/D16-1264> . <https://aclanthology.org/D16-1264>
- [13] Rajpurkar, P., Jia, R., Liang, P.: Know what you don’t know: Unanswerable questions for SQuAD. In: Gurevych, I., Miyao, Y. (eds.) Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), pp. 784–789. Association for Computational Linguistics, Melbourne, Australia (2018). <https://doi.org/10.18653/v1/P18-2124> . <https://aclanthology.org/P18-2124>